

Metric	Operational significance
CPU utilisation	Compute demand relative to provisioned capacity
Memory utilisation	Buffer pool saturation and working-set pressure
Active/Executing connections	Concurrency load relative to configured maximum (specifically isolating active execution from idle pooled connections)
Disk IOPS	Storage throughput pressure and I/O ceiling proximity

The model is trained on the joint distribution of these four metrics over a rolling four-week window — creating a 16-dimensional input vector per instance for every classification cycle.

This multi-dimensional structure enables the model to distinguish compound risk profiles that single-metric analysis cannot separate. A database running at 75% CPU with 20% active connection utilisation is healthy. The same CPU reading paired with 94% connection utilisation is often a precursor to an incident. The model learns this distinction

directly from historical outcomes. No static threshold rule can encode it reliably.

The open source engine: From telemetry to orchestration

The framework in Figure 1 is structured across four discrete layers, each with a clearly defined operational function and independent replaceability.

Layer 1 — Observability: Prometheus becomes the nervous system — scraping `postgres_exporter` and `node_exporter` at 15-second intervals, 24/7, without touching application code or requiring schema changes. The `postgres_exporter` exposes internal PostgreSQL statistics directly. `Node_exporter` provides host-level CPU and memory metrics. VictoriaMetrics serves as the long-term time-series store, retaining five weeks of data and answering PromQL range queries with efficient compression.

Layer 2 — Intelligence: This is where raw telemetry becomes decisions. Feature engineering produces a 16-dimensional input vector for each database instance — the behavioural fingerprint. A gradient boosting classifier trained on labelled historical data learns to generalise the classification decision. MLflow manages the model lifecycle — tracking

experiments, versioning models, and maintaining an audit trail that maps every weekly recommendation to the exact model version and input features. This auditability is essential for the human review phase.

Layer 3 — Orchestration: Apache Airflow doesn't just schedule the pipeline — it enforces the human review gate. Each stage of the weekly directed acyclic graph is independently retrievable and fully logged. In the first 60 days, every recommendation lands in an engineer's inbox before a single resource changes. This gate surfaces recommendations to the infrastructure team, builds operational confidence, and provides corrections that improve subsequent retraining.

Layer 4 — Execution: Three execution paths accommodate different PostgreSQL deployment topologies:

- **Kubernetes VPA:** Automatically adjusts CPU and memory resource requests on PostgreSQL pods during their next scheduled lifecycle event.
- **CloudNativePG Operator:** Preserves cluster topology and replication while applying resource changes through standard Kubernetes patch operations.
- **Patroni REST API:** Implements live configuration updates like `work_mem`

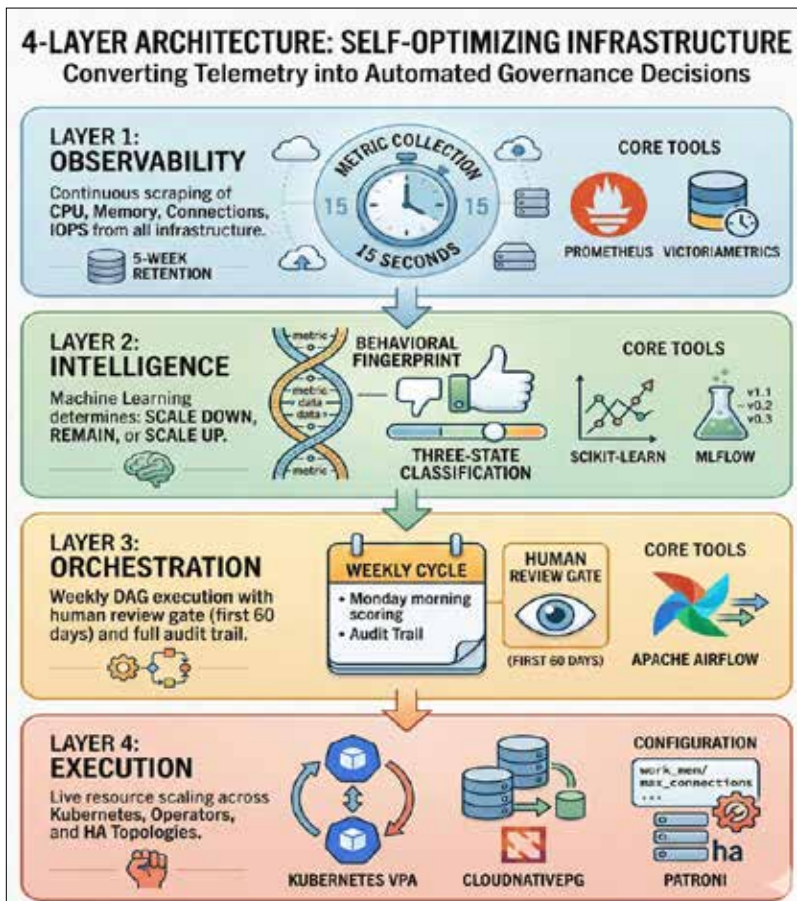


Figure 1: Self-optimising infrastructure